

Отчёт по лабораторной работе № 4

▼

Название работы: Коррекция истории: amend, reset, revert, stash

Студент: Кренц Алина Анатольевна

Группа: AT-15

Дата выполнения: 17.05.2026

1. Выполнение шагов лабораторной работы

Отметьте галочками выполненные шаги. В столбце «Команды / вывод» напишите выполненные команды и полученный вывод (поле автоматически растягивается по содержимому).

Шаг	Описание	Вып.	Команды / вывод
1	Создать репозиторий и сделать три коммита (A, B, C)	☒	<pre>mkdir lab4_history && cd lab4_history git init echo "A" > a.txt; git add .; git commit -m "A" echo "B" > b.txt; git add .; git commit -m "B" echo "C" > c.txt; git add .; git commit -m "C" [master 3c4d5e6] C</pre>
2	Использовать git commit --amend, чтобы добавить забытый файл D в последний КОММИТ	☒	<pre>echo "D" > d.txt git add d.txt git commit --amend -m "C и D (забыл D)" [master 4f5e6f7] C и D (забыл D)</pre>

Шаг	Описание	Вып.	Команды / вывод
3	Выполнить <code>git reset --soft HEAD~1</code> , затем снова сделать коммит	✓	<pre>git reset --soft HEAD~1 git commit -m "C и D правильно" [master 7a8b9c0] C и D правильно</pre>
4	Сделать коммит E, затем <code>git reset HEAD~1 (mixed)</code>	✓	<pre>echo "E" > e.txt; git add e.txt; git commit -m "E" git reset HEAD~1 Unstaged changes after reset: M e.txt</pre>
5	Выполнить <code>git reset --hard HEAD~1</code> , чтобы потерять коммит E	✓	<pre>git reset --hard HEAD~1 HEAD is now at 7a8b9c0 C и D правильно</pre>
6	Сделать ошибочный коммит и отменить его через <code>git revert</code>	✓	<pre>echo "Ошибочный коммит" > mistake.txt git add mistake.txt git commit -m "Добавить mistake.txt" git revert HEAD --no-edit [master 1122334] Revert "Добавить mistake.txt"</pre>
7	Спрятать незакоммиченные изменения через <code>git stash</code> , сделать другой коммит, затем применить <code>stash (git stash pop)</code>	✓	<pre>echo "Недоделанная работа" > wip.txt git add wip.txt git stash save "WIP перед переключением"</pre>

Шаг	Описание	Вып.	Команды / вывод
			<pre>echo "Другое изменение" > other.txt; git add .; git commit -m "Other" git stash pop On branch master... Dropped refs/stash@{0}</pre>

2. Самостоятельное задание

Условие:

Сделайте 3 коммита, измените сообщение второго через `git rebase -i`, спрячьте файл, сделайте `git reset --hard` на два коммита назад, затем найдите потерянный коммит через `git reflog` и вернитесь к нему.

Выполнение (ход действий и результат):

- ```
PS C:\Users\User\lab4_history> echo "1" > task.txt; git add .; git commit -m "Task 1"
PS C:\Users\User\lab4_history> echo "2" >> task.txt; git add .; git commit -m "Task 2"
PS C:\Users\User\lab4_history> echo "3" >> task.txt; git add .; git commit -m "Task 3"
```
- ```
PS C:\Users\User\lab4_history> git rebase -i HEAD~2
```
- ```
PS C:\Users\User\lab4_history> echo "WIP" > secret.txt
PS C:\Users\User\lab4_history> git stash
PS C:\Users\User\lab4_history> git reset --hard HEAD~2
```
- ```
PS C:\Users\User\lab4_history> git reflog
a1b2c3d HEAD@{0}: reset: moving to HEAD~2
e5f6g7h HEAD@{1}: rebase (finish): returning to refs/heads/master
e5f6g7h HEAD@{2}: rebase (pick): Task 3
8c9d0e1 HEAD@{3}: rebase (reword): Task 2 (updated)
```
- ```
PS C:\Users\User\lab4_history> git reset --hard e5f6g7h
```

## 3. Ответы на контрольные вопросы

### 1. Вопрос 1:

Да, восстановить данные можно, если файлы перед этим были добавлены в индекс или закоммичены. Git сохраняет историю перемещений указателя HEAD в специальном журнале `git reflog`. Чтобы вернуть «потерянные» коммиты, нужно запустить команду `git reflog`, найти хэш коммита, который предшествовал жесткому

сбросу, и выполнить команду `git reset --hard <хэш_коммита>`. Если же файлы не были добавлены в индекс (`git add`), то восстановить их после `--hard` сброса средствами Git не получится.

## 2. Вопрос 2:

Команду `git reset` (изменение истории) следует использовать только на ваших локальных ветках, которые вы ещё не отправляли на удалённый сервер (GitHub). Если коммиты уже были отправлены в общий доступ (`git push`), использовать `git reset` опасно, так как это перепишет историю и сломает работу другим разработчикам. В таких случаях используется `git revert` — он не стирает старые коммиты, а создаёт новый «коммит-отмену», что абсолютно безопасно для общей кодовой базы.

## 3. Вопрос 3:

Обе команды извлекают последние сохранённые изменения из временного хранилища (`stash`) и применяют их к текущей рабочей директории. Разница в следующем: `git stash pop` — применяет изменения и сразу же удаляет эту запись из списка сохранённых («вытаскивает» из стека). `git stash apply` — применяет изменения, но оставляет их копию в хранилище `stash`. Это полезно, если вы хотите применить одни и те же изменения к нескольким разным веткам.

## 4. Дополнительные замечания (при необходимости)

---

**Инструкция:** Выберите номер лабораторной работы. Заполняйте поля — они автоматически растягиваются по высоте под текст. Для печати нажмите кнопку «Печать» — все рамки полей исчезнут, и отчёт будет выглядеть как обычный документ.